

Formal and Empirical Analysis of Authorization Decision Inconsistency in SAGA

Abstract

We analyze an authorization decision inconsistency recorded in SAGA. Direct matcher calls return permission when a specific denial precedes wildcard permission, although most-specific semantics require denial; reversing the rules changes the implementation result. Independent finite specification evaluation and a checked observation supply a fixed symbolic scenario. Under explicit assumptions, ProVerif reports message acceptance reachable and establishes its correspondence to prior token issuance. Separately, A/A, D/D, and D/A consumer experiments execute production authorization logic in a controlled environment. D/A reaches one-time-key release and receiver token issuance, sending, and storage. The empirical endpoint precedes message acceptance. These results establish neither a deployed-service exploit nor Python/ProVerif refinement.

1 Introduction

SAGA [12] governs communication between registered agents using contact policies, identity checks, and access-control tokens. Authentication establishes who is communicating; it does not establish whether the receiving user’s policy permits that contact. We present a formal and empirical analysis of a recorded authorization decision inconsistency in SAGA.

Direct calls to the unmodified matcher show that a specific denial followed by wildcard permission returns an allow-side budget, whereas reversing the same rules returns denial. Documented most-specific semantics require denial in both orders. We analyze the conditional consequences of this disagreement in a fixed symbolic scenario: ProVerif reports message acceptance reachable and establishes that acceptance implies prior token issuance. Separately, controlled consumer experiments execute the production Provider and receiver through token issuance and storage, stopping before message acceptance. These are distinct formal and empirical results.

Our case-study contributions are:

- **Recorded decision inconsistency analysis.** We characterize the recorded disagreement through source inspection, direct matcher calls, and independent finite specification evaluation.
- **Consumer-side propagation validation.** A/A, D/D, and D/A specification/implementation controls connect fresh matcher observations to the production authorization workflow through token issuance and storage in a controlled environment.
- **Conditional formal consequence analysis.** We analyze reachability and prior-issuance correspondence under explicit symbolic assumptions, with independent gate controls distinguishing modeled release and issuance boundaries.

Related work. Agent research addresses autonomous behavior, trust, and prompt injection [13, 11, 5]. Our focus is policy evaluation and its consumers. Authentication and distributed authority have longstanding foundations [9]; SPKI, OAuth, and Macaroons address credentials and delegation [6, 8, 3]. Cassandra, SecPAL, and Margrave address policy semantics and analysis [2, 1, 7], while ProVerif and Tamarin support symbolic protocol analysis [4, 10]. This study concerns a concrete implementation disagreement and its conditional consequences, rather than a new policy language or general verification method.

2 Authorization Decision Inconsistency

Authorization flow and policy semantics. An initiating agent requests contact with a receiving agent through SAGA’s Provider. The receiving user’s ordered contact rulebook supplies patterns and integer budgets. Both the Provider and receiver invoke the shared matcher against the initiating identity. The Provider’s policy check precedes contact-material release; the receiver re-evaluates its local policy before the handshake can issue a token. Material release, token issuance, and subsequent message acceptance are distinct boundaries.

For a fixed rulebook P_R and valid initiating identity I , the documented contract selects the most specific matching rule, using `aid_specificity()`. This numerical score is not a general pattern-language containment ordering.

We consider a unique highest-scoring match with budget b^* , or no match with $b^* = 0$, and define

$$\text{Allow}_{\text{spec}}(P_R, I) \iff b^* > 0.$$

The positive-budget convention follows the Provider’s `/access` check. It describes policy eligibility, not sufficient conditions for a successful handshake. Equal-score conflicts and changing policies are outside this case. The consumer experiments use only negative and positive budgets; the Provider and receiver differ at zero, which is not evaluated as a propagation case here.

Root cause. In the inspected matcher, `best_pattern` is initialized to `None` but never updated when a matching rule changes the returned budget. Later comparisons therefore use the sentinel score -1 rather than the score of the best previous match. Normally scored matching patterns have nonnegative scores, so each passes the comparison and the last matching rule supplies the result. The relevant order is the rule list order, retained by registration, Provider storage, and Agent loading. The mismatch concerns the studied revision, not current upstream status.

Recorded matcher evidence. The direct-call archive records a valid rulebook in `policy-matcher-sanity.json` for identity `alice@example.com:agent`. Let s denote `alice@example.com:*` with budget -1 and w denote `*` with budget 10 . Their specificity scores are 70 and 0 . Table 1 reports direct calls to the unmodified production matcher, rather than the separate illustrative matcher script.

Rule order	Expected budget	Observed budget
Specific deny, then wildcard allow	-1	10
Wildcard allow, then specific deny	-1	-1

Table 1: Archived direct matcher observations for the same rule set.

An independent finite specification evaluator selects the specific denial for the recorded ordered input, without using the archive’s expected-result field as its answer. Writing

$$\text{Allow}_{\text{impl}}(P_R, I) \iff \text{match}(P_R, I) > 0,$$

the specific-deny/wildcard-allow order yields $\text{Allow}_{\text{impl}} \neq \text{Allow}_{\text{spec}}$. The reversed-order observation establishes order dependence without any equal-score ambiguity. These function calls establish the local decision inconsistency; Section 4 separately examines consumer execution.

The direct-call record uses Python 3.14.3. Its archive is `proofs/evidence/authz-stage1-20260916T123834944479Z/`. Its source revision and observation are bound separately from the fresh consumer run and the symbolic analysis.

3 Formal Consequence Analysis

Decision input. The bridge is a provenance-checked transformation component for this case study. It checks the recorded ordered rules, initiating identity, and historical matcher provenance, and combines independent finite specification evaluation with the archived implementation observation:

$$(D_{\text{spec}}(P, I), D_{\text{impl}}^{\text{obs}}(P, I)) = (\text{Deny}, \text{Allow}).$$

The supported evaluator grammar covers the literal and wildcard patterns needed for this case; it is not complete Python or `fnmatch` semantics. The checked pair reconstructs only the scenario region of a fixed consequence template. Protocol processes and queries remain unchanged; symbolic role mapping remains an analysis choice.

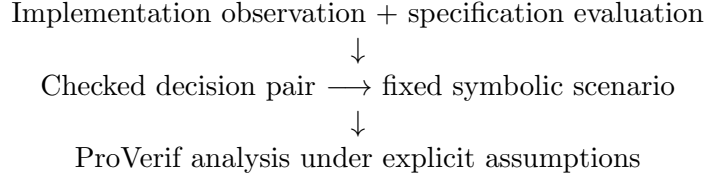


Figure 1: Case-specific decision transfer; the arrows do not establish implementation refinement.

Table 2 gives the scenario facts. ProVerif consumes these fixed inputs; it neither executes the matcher nor discovers its inconsistency.

Checked decision	Symbolic table fact
Specification: Deny	<code>ScenarioSpecDeny(BuggyPolicy, aid.B)</code>
Observation: Allow	<code>ScenarioImplAllow(BuggyPolicy, aid.B)</code>

Table 2: Decision values transferred into the fixed scenario.

Model and assumptions. The initiating peer I has a genuine registered identity and an existing receiving-user policy outside its control. The scenario assumes the same fixed policy at the Provider and receiver, effective

cryptographic checks, and no compromise of the Provider, CA, or honest private keys. The prescribed initiating-peer process does not represent arbitrary adversary-controlled application behavior.

The Provider releases contact material; **PeerA** represents receiver R , which issues tokens and accepts messages; **PeerB** represents initiator I . A CA supplies certificates. Replicated processes use symbolic cryptography, and the private `tls_chat` channel abstracts authenticated, confidential transport. The **ProviderReleased** table abstracts handoff to receiver processing; **ActiveToken** associates the participants, token, and sender’s access-control public key.

SpecIntendedDeny(I, R) records scenario denial without blocking execution. **ProviderRelease**(R, I) records release, **TokenIssue**(R, I, t) records wire-token issuance, and **ChatAccept**(R, I, t, m) records application-message acceptance. The latter is distinct from **Accept**, which denotes token receipt and verification in the diagnostic model.

Queries and results. The archived ProVerif 2.05 execution of the bridge-derived scenario reports the two results in Table 3. A reconstructed witness refutes `not event(ChatAccept(...))`; acceptance is existentially reachable under the supplied divergence. The non-injective correspondence

$$\text{ChatAccept}(R, I, t, m) \Longrightarrow \text{TokenIssue}(R, I, t)$$

requires prior issuance for the same participants and token, not intended policy permission or a unique session. Issuance occurs in the witness; there is no standalone issuance-reachability query in this model.

Query in the derived-model run	Result
ChatAccept reachability	Reachable
ChatAccept $\Rightarrow$ TokenIssue	True

Table 3: Archived formal query outcomes for the fixed D/A scenario.

The run is archived under `proofs/evidence/authz-bridge-turepass-20260922T111657792091Z/`. Its manifest binds the observation, generated model, verifier, and outputs. It is separate from the historical preset-divergence run and the consumer experiment; agreement of outcomes does not identify a common execution.

The earlier Stage 1 diagnostic retains the queried authentication correspondences and token secrecy, while $\text{Accept}(R, I, t) \Rightarrow \text{PolicyAllow}(R, I)$ is

false. Because that process never emits `PolicyAllow`, this is a verification-coverage diagnostic, not formal detection of the matcher defect. Those guarantees do not automatically transfer to the consequence model.

Independent gate controls. The existing controls in Table 4 isolate modeled blocking positions. Their labels denote Provider/receiver permissions, not specification/implementation decisions. They use a fixed private token, unlike the fresh-token divergence model, and are not verified repairs or refinements of it.

Provider	Receiver	Release	Issue	Accept
Allow	Allow	R	R	R
Deny	Allow	U	U	U
Allow	Deny	R	U	U

Table 4: Independent gate controls: reachable (R) and unreachable (U).

In addition to the prior-issuance correspondence, the controls establish

$$\text{ChatAccept}(R, I, t, m) \Rightarrow \text{ChatSend}(R, I, t, m).$$

Both correspondences are vacuous when acceptance is unreachable. Provider denial blocks release and downstream events; receiver denial blocks issuance and acceptance while release remains reachable. The evidence remains in `proofs/evidence/authz-chat-gates-20260918T023621017067Z/`.

4 Consumer-Side Propagation Validation

Purpose. Phase 1 tests whether fresh matcher observations propagate through the production consumer-side authorization workflow to token issuance and storage in a controlled environment. This experiment is separate from the symbolic message-acceptance result in Section 3.

Environment. The harness executes the production Provider `/access` route with a real isolated MongoDB instance and passes the returned one-time key (OTK) to the production receiver handshake. Both consumers call the production matcher through observers preserving inputs, return values, and exceptions. The receiver re-evaluates its policy; no single decision object is transmitted between consumers.

Production signature checks, OTK consumption, key derivation, token encryption, and `active_tokens` storage remain in the executed path. Registration state and certificate retrieval use fixtures. A Flask test client and an in-memory connection replace network transport and TLS; external registration and certificate services are not exercised. The sole execution cutoff is `receive_conversation`, after the token has been stored and sent. The stub records this boundary without executing conversation logic or `local_agent.run`. The archived environment uses Python 3.12.7 and MongoDB 8.0.17 on Windows.

Experiments. Each case starts with fresh cryptographic material, empty counter and token stores, and one available OTK. The initiator is `alice@example.com:agent`; the fixture receiver is `bob@example.com:receiver`. Provider and receiver hold identical ordered policies. Let s denote `alice@example.com:*` and w denote `*`. A/A uses $[(s, 10), (w, 10)]$; D/D uses $[(w, 10), (s, -1)]$; D/A uses $[(s, -1), (w, 10)]$. The first label is the independent specification decision, and the second is the observed implementation decision. The harness reuses the finite evaluator read-only, without importing historical matcher observations. Each case ran once.

Spec./impl.	Provider HTTP	OTK released	Token sent/stored
A/A	200	Yes	Yes
D/D	403	No	Not reached
D/A	200	Yes	Yes

Table 5: Consumer propagation results; message acceptance was not executed.

A/A is the positive control demonstrating the permitted path. D/D is the Provider-blocking control: HTTP 403 leaves state unchanged, and the receiver is not reached. It does not validate receiver rejection. D/A tests propagation of the observed decision inconsistency.

Result and evidence. In D/A, the specification selects budget -1 , but both calls to the matcher return 10. The Provider returns HTTP 200, releases its OTK, and initializes the counter to 9. The response, receiver handshake, and initial OTK state share the same key hash; the receiver consumes that key. A single token is entered in `active_tokens` and sent, with matching token hashes across the send record, boundary observation, and stored state. Initiator-side decryption agrees with the stored token metadata, including the recipient key and initial quota. Thus the observed workflow

reaches OTK release, receiver processing, and token issuance, sending, and storage.

The successful archive is:

```
proofs/consumer_validation/evidence/20260926T0700219  
21746Z-d64c8064/.
```

Its `manifest.json` and `run-start.json` identify the run, environment, substitutions, actual source-byte hashes, and status. Hash-bound per-case inputs, event logs, and results link policy decisions to HTTP responses and state changes. The run records its dirty working tree rather than claiming a clean revision; these fresh observations are separate from the historical matcher record, formal runs, and failed harness archives.

The endpoint is issuance/storage, not message acceptance. No application message, LLM call, or tool execution occurs. One execution per case is a bounded functional observation, not a reliability or performance study.

5 Scope and Conclusion

The evidence concerns the recorded SAGA case and tested policies, not arbitrary policies, current upstream vulnerability status, or prevalence across deployments. The finite specification evaluator is not theorem-proved. Formal results depend on symbolic role mapping, transport, and handoff assumptions; they do not establish token expiry, quota consumption, revocation, policy updates, session isolation, or injective agreement. Consumer validation uses controlled registration and transport fixtures and stops before message acceptance. Neither the symbolic witness nor the consumer experiment establishes a deployed-service exploit, Python/ProVerif refinement, or a verified production repair.

The recorded rule-order inconsistency separates intended policy permission from the matcher’s observed decision. Conditional symbolic analysis admits message acceptance with prior token issuance, while the independent consumer experiment observes propagation through real authorization logic to token issuance and storage. Authentication and credential provenance therefore do not substitute for decision correctness in this case. Future work may investigate stronger implementation-to-model correspondence and additional systems.

References

- [1] Moritz Y. Becker, Cédric Fournet, and Andrew D. Gordon. Design and semantics of a decentralized authorization language. In *20th IEEE Computer Security Foundations Symposium (CSF)*, pages 3–15. IEEE, 2007.
- [2] Moritz Y. Becker and Peter Sewell. Cassandra: Distributed access control policies with tunable expressiveness. In *5th IEEE International Workshop on Policies for Distributed Systems and Networks (POLICY)*, 2004.
- [3] Arnar Birgisson, Joe Gibbs Politz, Úlfar Erlingsson, Ankur Taly, Michael Vrabie, and Mark Lentzner. Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud. In *Network and Distributed System Security Symposium (NDSS)*. Internet Society, 2014.
- [4] Bruno Blanchet. Modeling and verifying security protocols with the applied pi calculus and ProVerif. *Foundations and Trends in Privacy and Security*, 1(1–2):1–135, 2016.
- [5] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems*, volume 37, 2024.
- [6] C. Ellison, B. Frantz, B. Lampson, R. Rivest, B. Thomas, and T. Ylonen. SPKI certificate theory. RFC 2693, RFC Editor, September 1999. Experimental.
- [7] Kathi Fisler, Shriram Krishnamurthi, Leo A. Meyerovich, and Michael Carl Tschantz. Verification and change-impact analysis of access-control policies. In *27th International Conference on Software Engineering (ICSE)*, 2005.
- [8] D. Hardt. The OAuth 2.0 authorization framework. RFC 6749, RFC Editor, October 2012. Editor; Standards Track.
- [9] Butler Lampson, Martín Abadi, Michael Burrows, and Edward Wobber. Authentication in distributed systems: Theory and practice. *ACM Transactions on Computer Systems*, 10(4):265–310, 1992.

- [10] Simon Meier, Benedikt Schmidt, Cas Cremers, and David Basin. The TAMARIN prover for the symbolic analysis of security protocols. In *Computer Aided Verification (CAV)*, volume 8044 of *Lecture Notes in Computer Science*, pages 696–701. Springer, 2013.
- [11] Sarvapali D. Ramchurn, Dong Huynh, and Nicholas R. Jennings. Trust in multi-agent systems. *The Knowledge Engineering Review*, 19(1):1–25, 2004.
- [12] Georgios Syros, Anshuman Suri, Jacob Ginesin, Cristina Nita-Rotaru, and Alina Oprea. SAGA: A security architecture for governing AI agentic systems. arXiv preprint arXiv:2504.21034v2, 2025. Version 2, revised 29 August 2025.
- [13] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Jirong Wen. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024.